EC 504
Final Exam Practice, Fall 2025

Note: This is much longer than the final exam will be, but it has questions covering all of the possible topics in the exam. Use it for practice. The real exam will be shorter.

Problem 1

Answer True or False to each of the questions below, **AND YOU MUST PROVIDE SOME FORM OF EXPLANATION (very brief is OK)**. Each question is worth 2 points. Answer true only if it is true as stated, with no additional assumptions.

(a) In a forward star representation of a directed graph, the edges must be sorted by start node.

Solution:

True; that is the definition of a forward star.

(b) Suppose T_1, T_2 are minimum spanning trees for the same graph G. Then the largest edge weight of T_1 must be the same weight as the largest edge weight of T_2.

Solution:

TRUE. Otherwise, can find a contradiction by taking the largest of the two largest weights out of that tree and replacing it with an edge from the other tree.

(c) In Bellman-Ford's shortest path algorithm, the temporary distance labels D assigned to vertices which have not yet been scanned (i.e. left the work queue) can be interpreted as the minimum distance among all paths from the start vertex to the given vertices with intermediate vertices restricted to the vertices that have already been scanned (left the work queue).

Solution:

False. That is Dijkstra's algorithm.

(d) Consider a minimum spanning tree T in a weighted, undirected graph G. Suppose we decrease the weight of a single edge that is in the minimum spanning tree T. Then, it is possible that T is no longer a minimum spanning tree in the modified graph.

Solution:

False. Kruskal's algorithm would try to insert this edge into the minimum spanning tree earlier, but it would find the same edges.

(e) Consider a minimum spanning tree problem in a connected, weighted graph with positive and negative weights which can include negative weight cycles. Then, both Prim's and Kruskal's algorithms can be used to find the minimum weight spanning tree in this graph.

Solution:

TRUE. The algorithms do not care as to whether the weights are positive or negative.

(f) In the KMP algorithm for string matching, the prefix function for the string "AAABBBA" is [0,1,2,0,1,2,1].

Solution:

False. It is [0,1,2,0,0,0,1].

(g) Consider a minimum spanning tree T in a weighted, undirected graph G. Suppose we decrease the weight of a single edge **that is in the minimum spanning tree** T. Then, it is possible that T is no longer a minimum spanning tree in the modified graph.

Solution:

False. Kruskal's algorithm would try to insert this edge into the minimum spanning tree earlier, but it would find the same edges.

(h) A problem is said to be NP-complete if it can be reduced in polynomial time to an instance of every other problem which is in class NP.

Solution:

False. Every problem in NP can be reduced in polynomial time to an instance of this problem.

(i) Given a weighted, directed, acyclic graph with non-negative weights, consider the problem of finding the longest path from node 1 to every other node. We can solve this problem using the Bellman-Ford algorithm.

Solution:

True. Changing all the distances to negative numbers, we can find the shortest paths because the graph is acyclic so there are no negative cost cycles.

(j) Consider a minimum spanning tree T in a connected, weighted undirected graph (V, E) where the weights have different values. Then, for every node i, the minimum spanning tree must contain the arc $\{i, j\}$ with the smallest weight connected to node i.

Solution:

True. Otherwise, we can add that arc, and disconnect a current arc adjacent to node i, and get a smaller weight spanning tree.

(k) Dynamic programming can be used to solve the integer knapsack problem in pseudo-polynomial time.

Solution:

True. $O(nC)$.

(l) The sequence alignment problem can be solved using dynamic programming in time polynomial in the lengths of the two sequences to be aligned.

Solution:

True. $O(mn)$,where m, n are the lengths of the sequences.

(m) Consider a 2-d binary search tree that was constructed by branching on the median value at each level, storing the keys at the leaves of the tree. Assume there are n keys in the tree, where each key is an ordered pair of values. The worst case complexity to find a key is $O(\log(n))$.

Solution:

True. This is for balanced trees.

(n) Consider an undirected graph where, for every pair of nodes i, j, there exists a unique simple path between them. This graph must be a tree.

Solution:

True. This is an alternative definition of a tree, since it implies there is one and only one simple path. This implies every pair of nodes is connected, and that there are no cycles.

(o) Prim and Kruskal's algorithm can also be used to find the maximum weight spanning tree in a graph.

Solution:

True. To find the maximum weight, simply compute the negative value of the weights and find the minimum weight. Note that you can always add a constant to every arc, and get the same minimum spanning tree, since the number of arcs is the same for every spanning tree. Thus, you can solve the problem with negative weight arcs.

(p) Consider a connected, undirected graph. Then, the worst case computational complexity of the Floyd-Warshall algorithm for finding the shortest distance for every pair of nodes does not depend on the number of edges in the graph.

Solution:

True. It only depends on the number of nodes.

Problem 2

Consider the following 2-dimensional set of points:

$$(80, 90), (85, 78), (60, 40), (10, 10), (15, 15), (30, 80)$$

(a) Arrange the elements in a PR quadtree with value range 0-96 for both the first and second coordinates. Display the answer **as a tree**, with children arranged left-to-right in the following order: SW, SE, NW, NE.

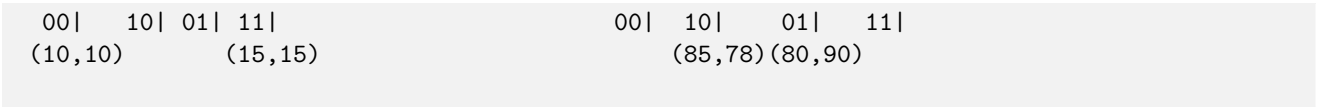
Solution:

```
Sorting to coarsest level, we get nodes that need expansion
               48,48
                 |
         ------------------------
     00|    01|      |10       |11
   (10,10) (30,80) 60,40    (80,90)
   (15,15)                  (85,78)



We now subdivide to next level: squares of 24 x 24
                          48|48
                            |
                 -------------------------
              00|    10|       |01        |11
              24|24 (60,40) (30,80)     72|72
                |                         |
          ---------------          -----------------
       00|   10|  |01  |11       00|   10|  |01   |11
     (10,10)                                 (80,90)
     (15,15)                                 (85,78)

We now subdivide to next level: squares of 12 x 12
                          48,48
                            |
                 -------------------------
              00|    01|      |10         |11
              24|24 (30,80) 60,40       72,72
                |                         |
          ---------------          -----------------
       00|   10|  |01  |11       00|   10|  |01  |11
       12|12                                 84|84
    -------------                          ------------------
```



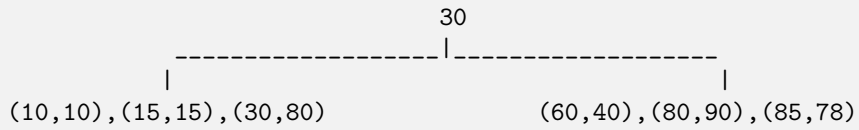
(b) Form a balanced 2-d tree using the median of the elements to split, with all the keys at the leaves of the tree. When there are an even set of points to split at vertex, select the smaller of the two possible median values as the navigation key.

Solution:

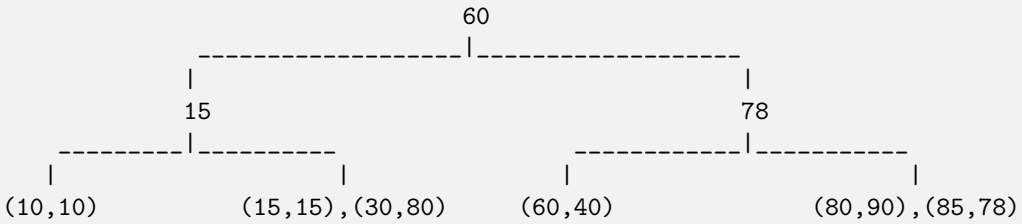
First, find the median of the first coordinates. Sorting by the first coordinate yields:

$$(10, 10), (15, 15), (30, 80), (60, 40), (80, 90), (85, 78)$$

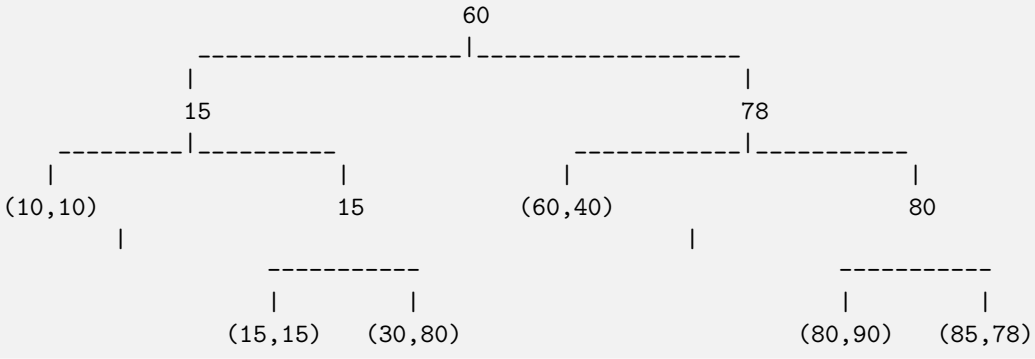
The median value is either 30 or 60, so we select the average 45 as indicated in the problem, resulting in



The median of the second coordinate on the right is 78, so we branch on 78. For the left list, the median is 15, so we branch on 15. We get:



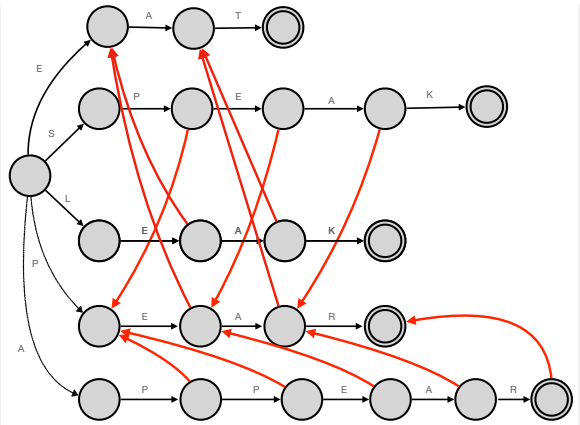
Repeat the process for each leaf with two entries, branching the average of the x-values.



Problem 3 Draw the Aho-Corasick trie corresponding to the following search patterns: SPEAK, LEAK, EAT, PEAR, APPEAR. Show the suffix links, except for the suffix links that revert back to the root. Don't bother showing the output links.

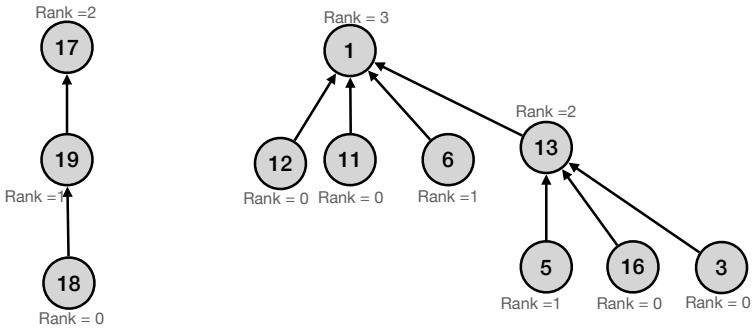
Solution:

The trie is shown on the right:



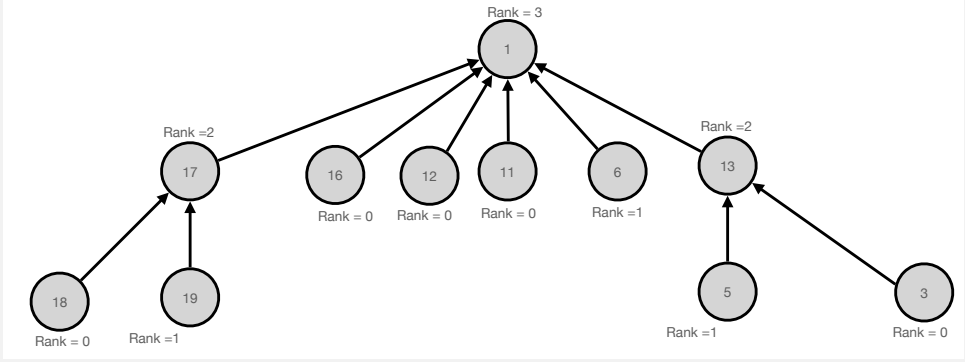
Problem 4

Consider the two trees, shown on the right, that are part of a disjoint set forest. Suppose we find a relation between keys 16 and 18. Show the disjoint set tree that results from the union operation using merge by rank and path compression, where, if two roots have the same rank, merge the larger value root under the smaller value root.



Solution:

The merged trees are shown below, using merge-by-rank and path compression.



Problem 5

Consider the two words: *people* and *purple*. Consider them as a sequence of letters. We would like to find the best alignment of the two letter sequences, with the following costs:

- Incur a penalty of 1 for any skipped letter.
- Incur a penalty of 3 for any mismatched letter .
- Incur a reward of 2 for correctly matching two letters.

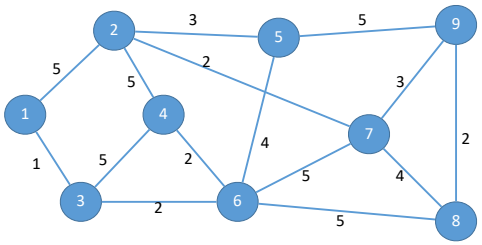
Using this reward structure, find the minimum cost alignment. State the cost of the alignment.

Solution:

	-	p	e	o	p	l	e	
-	0	1	2	3	4	5	6	
p	1	-2	-1	0	1	2	3	
u	2	-1	0	1	2	3	4	
r	3	0	1	2	3	4	5	
p	4	1	2	3	0	1	2	
l	5	2	3	4	1	-2	-1	
e	6	3	0	1	2	-1	-4	
	Match:	Cost -4						
	p	-	e	-	o	p	l	e
	p	u	-	r	-	p	l	e

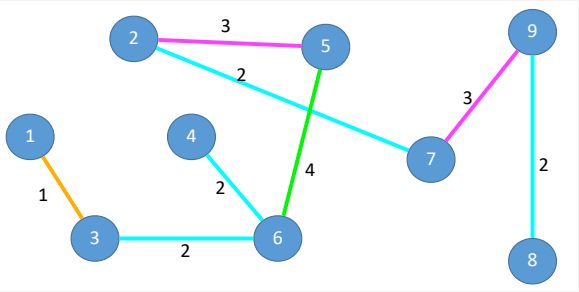
Problem 6

Consider the undirected graph on the right, where the numbers on edges consist of weights. Find a minimum spanning tree for the graph on the right. Indicate what algorithm you are using, and show some of the work associated with that algorithm.
Display the minimum spanning tree graphically, and show the total weight of the minimum spanning tree.



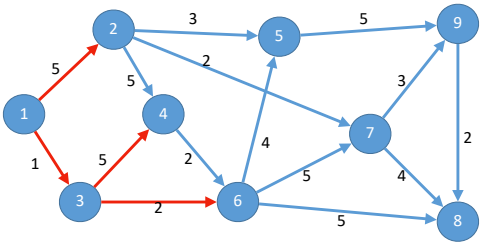
Solution:

Use Kruskal's. We start with the orange edge of weight 1, then the cyan edges of weight 2, then the magenta edges of length 3 (they don't form a cycle), then a green edge of length 4 (one of two possible solutions). Total weight 19. See below.

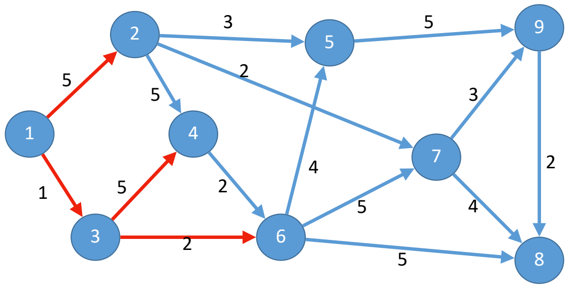


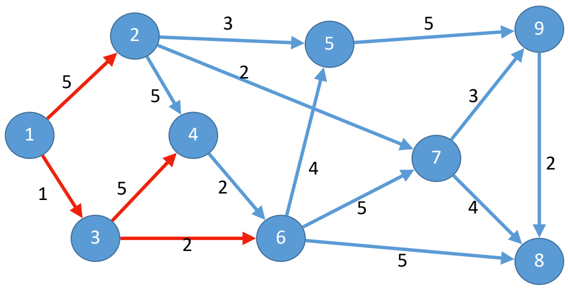
Problem 7

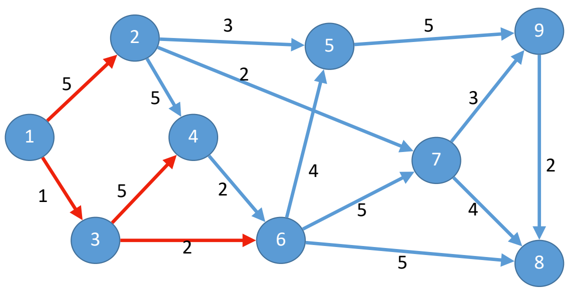
Consider the directed graph on the right, where the numbers on edges consist of distances. We are going to find a shortest path tree from node 1 to every other node using Dijkstra's algorithm. The figure shows the partial tree grown after five iterations, scanning vertex 1, then vertex 3, then vertex 6, then vertex 2, and then vertex 4.

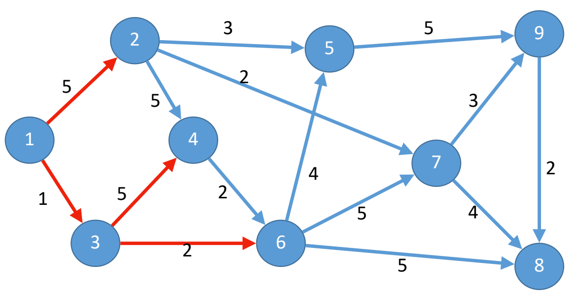


(a) Show the sequence of four paths that Dijstra's Algorithm will find to solve the problem, by drawing the edges on the graphs below.

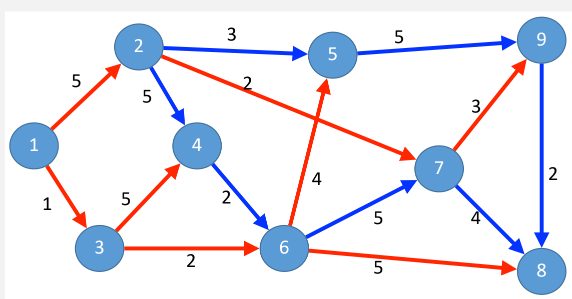






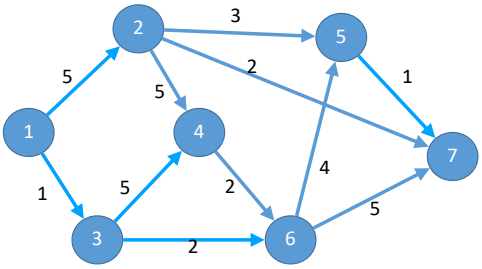


Solution:



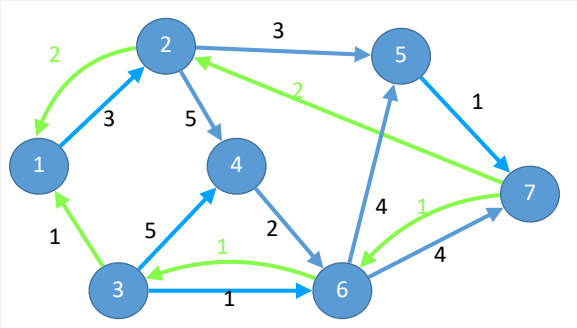
Problem 8

Consider the directed graph on the right, where the numbers on edges consist of edge capacities. Our goal is to find the max flow from vertex 1 to vertex 7. As such, we will do it using both the Ford-Fulkerson algorithm, using the Edmonds-Karp variation where shortest augmenting paths are found using breadth first search in the residual graph.



(a) Assume that we have made an augmentation of one unit of flow on path 1 - 3 - 6 - 7, and another augmentation of 2 units of flow on path 1 - 2 - 7. Construct the residual network, with residual capacities on each forward and reverse edge.

Solution:



(b) On that residual network, find the shortest (least number of hops) augmenting path from vertex 1 to vertex 7, and determine its capacity.

Solution:

The path is 1 - 2 - 5 - 7, capacity 1.

Problem 9

Consider the following knapsack problem: We have six objects with different values and V_i and sizes w_i. The object values and sizes are listed in the table below:

Object number	1	2	3	4	5	6
Value	10	11	12	13	14	15
Size	1	1	2	3	4	3

(a) Suppose we are given a knapsack with total size 11, and you are allowed to use fractional assignments. What is the maximum value that fits in the knapsack for the above tasks?

Solution:

Sort them by value per unit size in decreasing order, we get the following order: 1, 2, 3, 6, 4, 5.

The optimal fractional schedule will include all of objects 1, 2, 3, 6, 4 and 1/4 of object 5, for a total value of 64.5.

(b) Assume we are not allowed fractional assignments. For the knapsack with total size 11, the dynamic programming requires computing $Opt(j, k)$, the best value considering only objects $1, \ldots, j$, using total size k. The table below has partially computed these numbers. Compute the remaining elements in the table, $Opt(j, k), j = 5, 6; k = 5, 6, 7, 8, 9, 10, 11$.

Solution:

The table is

Capacity-> Max .Task	0	1	2	3	4	5	6	7	8	9	10	11
0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	10	10	10	10	10	10	10	10	10	10	10
2	0	11	21	21	21	21	21	21	21	21	21	21
3	0	11	21	23	33	33	33	33	33	33	33	33
4	0	11	21	23	33	34	36	46	46	46	46	46
5	0	11	21	23	33	34	36	46	47	48	50	60
6	0	11	21	23	33	36	38	48	49	51	61	62

The optimal integer value is 62, including objects 1, 2, 3, 5, 6.

(c) What is the optimal set of entire tasks that fit in the knapsack with size 11 and what is the maximum achieved value?

Solution:

The optimal integer value is 62, including objects 1, 2, 3, 5, 6.